

Q. List different threshold selection methods.

Ans: Thresholding is a basic image segmentation technique that converts a gray scale image into a binary image using a threshold value (T). Various methods are used to select an optimal threshold.

Threshold Selection Methods

- | | | |
|---|--|--|
| ① Fixed (Manual) Thresholding | : A threshold is chosen manually based on visual inspection or prior knowledge. | $T = \text{chosen manually}$ |
| ② Mean (Average) Thresholding | : Threshold is the mean of maximum and minimum intensity values. | $T = \frac{I_{\max} + I_{\min}}{2}$ |
| ③ Iterative Thresholding : (Ridler-Calvard) | : Starts with an initial threshold and iteratively updates using mean of two classes. | $T_{\text{new}} = \frac{\mu_1 + \mu_2}{2}$
Repeat until T converges |
| ④ Otsu's Method | : Selects threshold that maximizes between-class variance (or minimizes within-class variance). | $T = \arg \max_t \sigma_b^2(t)$ |
| ⑤ Entropy Based Thresholding | : Selects threshold that maximizes the information entropy between foreground and background. | $T = \arg \max_t [H_1(t) + H_2(t)]$ |
| ⑥ Kapur's Method (Max. Entropy) | : Threshold is selected by maximizing the sum of entropies of object and background. | $T = \arg \max_t [H_1(t) + H_2(t)]$ |
| ⑦ Maximum Likelihood Thresholding | : Assumes histogram follows two Gaussian distributions and selects threshold using maximum likelihood. | $T = \text{value that maximizes likelihood function}$ |
| ⑧ Clustering Based Thresholding (K-means) | : Histogram is clustered into two classes using K-means and threshold is chosen between cluster means. | $T = \frac{\mu_{c1} + \mu_{c2}}{2}$ |

Note: Otsu's method is widely used because it is automatic, simple and provides good results for bimodal histograms.

$I_{\max}$ - maximum intensity
 $I_{\min}$ - minimum intensity
 μ_1, μ_2 - means of two classes
 σ_b^2 - between-class variance
 H_1, H_2 - entropies of two classes

Apply Otsu's Thresholding Method to Segment an Image

Computer Vision and
Its Applications

1. Introduction

Otsu's thresholding is a global thresholding technique used to convert a grayscale image into a binary image. It automatically computes the optimal threshold that minimizes the intra-class variance (or maximizes the inter-class variance) between the foreground and background pixels.

2. Given

- Input : Grayscale image I of size $M \times N$ with L gray levels (0 to $L-1$)
- Output : Binary image I_b using optimal threshold T^*

3. Algorithm / Steps

Step 1 : Compute the normalized histogram

$$p_i = n_i / M \times N, \quad i = 0, 1, \dots, L-1$$

where n_i = number of pixels at gray level i .

Step 2 : Compute the cumulative sums

$$w_0(t) = \sum_{i=0}^t p_i \quad (\text{class 0 probability})$$

$$w_1(t) = \sum_{i=t+1}^{L-1} p_i = 1 - w_0(t) \quad (\text{class 1 probability})$$

Step 3 : Compute the cumulative means

$$\mu_0(t) = \sum_{i=0}^t i p_i / w_0(t) \quad \text{if } w_0(t) \neq 0$$

$$\mu_1(t) = \sum_{i=t+1}^{L-1} i p_i / w_1(t) \quad \text{if } w_1(t) \neq 0$$

Step 4 : Compute between-class variance

$$\sigma_b^2(t) = w_0(t) w_1(t) [\mu_0(t) - \mu_1(t)]^2$$

Step 5 : Find the threshold T^* that maximizes $\sigma_b^2(t)$.

$$T^* = \arg \max_{0 \leq t < L} \sigma_b^2(t)$$

Step 6 : Binarize the image using T^*

$$I_b(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T^* \\ 0 & \text{otherwise} \end{cases}$$

4. Pseudocode

```

for t = 0 to L-1
    compute  $w_0(t)$ ,  $w_1(t)$ ,  $\mu_0(t)$ ,  $\mu_1(t)$ 
    compute  $\sigma_b^2(t)$ 
end
 $T^*$  = threshold  $t$  with maximum  $\sigma_b^2(t)$ 
Generate binary image using  $T^*$ 
    
```

5. Example

Consider a 3-bit image ($L = 8$ gray levels)

Histogram (n_i):

Gray level (i)	0	1	2	3	4	5	6	7
n_i	10	15	20	25	30	20	10	5

Total pixels, $M \times N = 10 + 15 + 20 + 25 + 30 + 20 + 10 + 5 = 135$

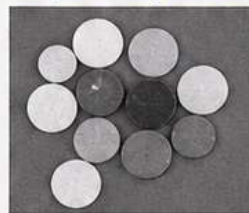
After computing $\sigma_b^2(t)$ for each t , we get maximum σ_b^2 at $t = 3$.
 $\approx$ Optimal threshold, $T^* = 3$

Binary image:

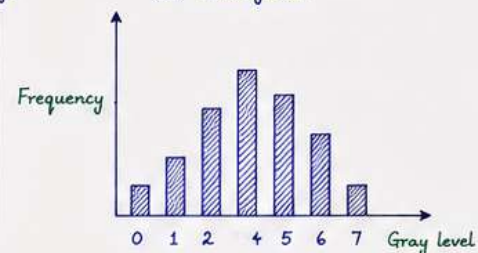
$$I_b(x, y) = 1 \quad \text{if } I(x, y) > 3 \quad \text{else } 0$$

6. Illustration

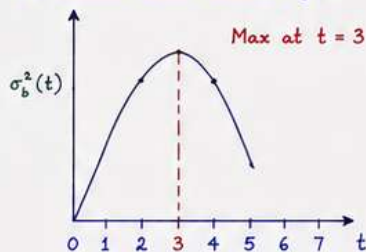
(a) Original Grayscale Image



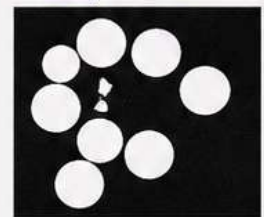
(b) Histogram



(c) Between-class variance $\sigma_b^2(t)$



(d) Segmented (Binary) Image using $T^* = 3$



7. Properties / Advantages

- Automatic selection of threshold (no manual tuning).
- Works well for bimodal histograms.
- Simple and computationally efficient.
- Provides good separation between foreground and background.

8. Applications

- Document image binarization
- Medical image segmentation
- Object extraction, quality inspection, etc.

Key Idea: Choose T that maximizes the separation between two classes.

Analyze the Effect of Illumination Changes on Threshold Selection

Computer Vision
and Its Applications

1. Introduction

Illumination (lighting) variations in an image alter the pixel intensities of both foreground and background. Since thresholding separates classes based on intensity, changes in illumination directly affect the choice of threshold and the quality of segmentation.

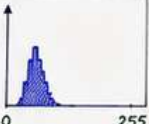
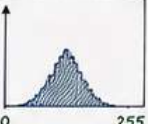
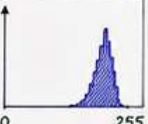
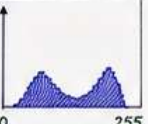
2. Types of Illumination Changes

- Increase in illumination (brighter image)
- Decrease in illumination (darker image)
- Non-uniform illumination (shadows, vignetting)
- Directional lighting (specular highlights, shading)

3. Effect on Image Histogram

Illumination Change	Histogram Effect	Effect on Threshold
Brighter	Histogram shifts to the right (higher gray levels)	Threshold increases
Darker	Histogram shifts to the left (lower gray levels)	Threshold decreases
Non-uniform	Histogram becomes broad or may be multimodal *	Single global threshold may fail
Directional	Creates long tails due to highlights/shadows	Global threshold not robust; local/adaptive threshold needed

4. Illustrative Example

Illumination	Underexposed (Dark)	Normal (Well lit)	Overexposed (Bright)	Non-uniform (Shadow)
Image				
Histogram				
Optimal Threshold T^*	Low	Medium	High	Varies / Local
Binary Result (using wrong T)				

5. Analysis

- When illumination increases, all pixel values increase. So, to separate foreground from background, the threshold must also increase.
- When illumination decreases, pixel values decrease and threshold must be lowered.
- Non-uniform illumination causes overlap between foreground and background intensities $\rightarrow$ global thresholding gives poor results.
- Adaptive (local) thresholding or illumination normalization helps in such cases.

6. Comparison of Thresholding Methods Under Illumination Changes

Method	Effect of Illumination Change	Robustness
Global Fixed Threshold	Highly sensitive; needs manual retuning for each change	Low
Otsu's Global Threshold	Automatically adapts to histogram shift; works well for uniform illumination	Medium
Adaptive Threshold	Computes local thresholds; handles non-uniform illumination better	High
Illumination Normalization	Reduces effect before thresholding (e.g., histogram equalization)	High

7. Remedies / Techniques

- Use adaptive (local) thresholding for non-uniform lighting.
- Apply histogram equalization or CLAHE to normalize illumination before thresholding.
- Use dynamic threshold selection methods (Otsu, entropy).
- Use background estimation and subtraction to reduce illumination effect.

8. Conclusion

Illumination changes significantly affect the histogram distribution and hence the selection of threshold. Proper thresholding requires adapting the threshold value according to the lighting conditions or using adaptive techniques to obtain accurate segmentation.

Key Takeaway

- Brighter image $\rightarrow$ Higher threshold
- Darker image $\rightarrow$ Lower threshold
- Non-uniform illumination $\rightarrow$ Use adaptive threshold
- Preprocessing (normalization) improves thresholding

State the Principle of the Canny Edge Detector.

COMPUTER VISION AND
ITS APPLICATIONS

1. Principle :

The Canny edge detector is based on optimal detection of edges by maximizing the signal-to-noise ratio. It uses a multi-stage algorithm and is designed to satisfy three optimality criteria.

2. Optimality Criteria (Principle)

① Good Detection (Low Error Rate)

- ✓ The detector should have a low probability of missing real edges (low false negative).
- ✓ It should also have a low probability of detecting non-edges (low false positive).

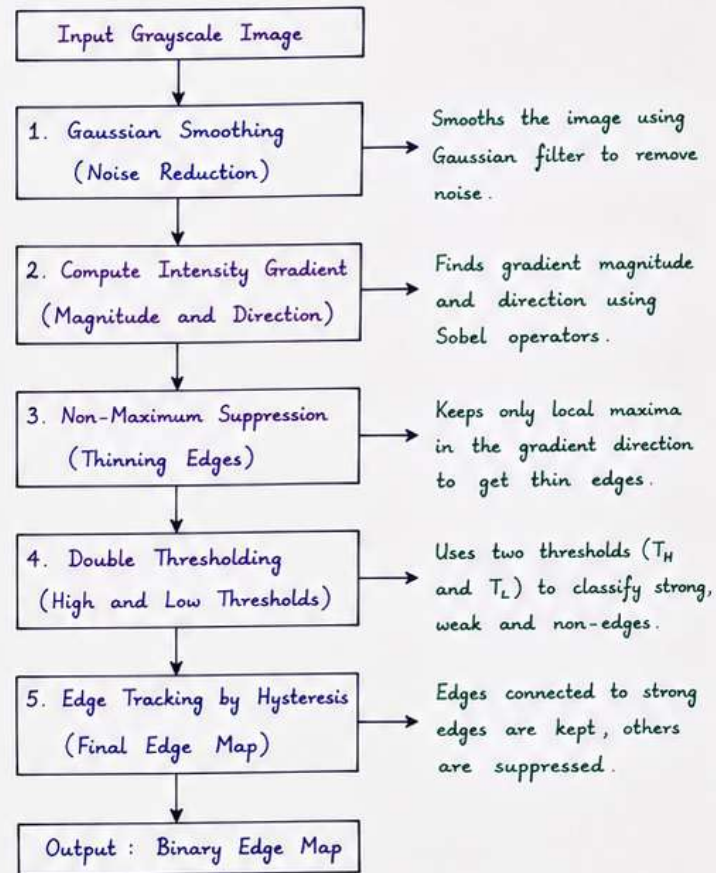
② Good Localization :

- ✓ The distance between the detected edge and the true edge should be minimum.
- ✓ Detected edges should be as close as possible to the actual edges.

③ Single Response :

- ✓ Each real edge should correspond to only one detected edge point.
- ✓ The detector should not produce multiple responses to a single edge.

3. Algorithm / Steps



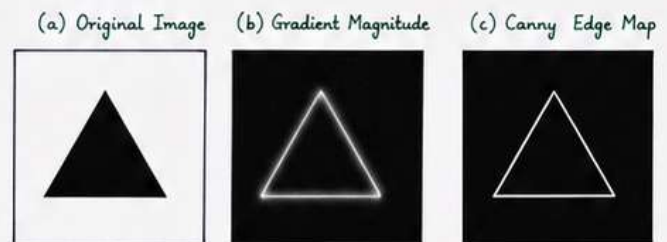
4. Explanation of Steps

1. Gaussian Smoothing : Reduces noise while preserving important features.
2. Gradient Computation : Finds intensity changes (edges) using derivatives.
3. Non-Maximum Suppression : Produces thin edges by suppressing non-maxima.
4. Double Thresholding : Classifies pixels into strong, weak and non-edges.
5. Edge Tracking by Hysteresis : Keeps weak edges only if they are connected to strong edges.

6. Key Features / Advantages

- Produces thin, well-localized edges.
- Less sensitive to noise due to Gaussian smoothing.
- Detects weak edges that are connected to strong edges.
- Widely used in object detection, segmentation and feature extraction.

5. Illustration



7. Summary

The Canny edge detector finds edges by maximizing detection, localization and single response. It uses Gaussian smoothing, gradient computation, non-maximum suppression, double thresholding and hysteresis to produce a clean, thin and reliable edge map.

★ Exam Point : The principle of Canny edge detector is to detect edges optimally by maximizing detection accuracy, minimizing localization error and ensuring single response.

Analyze the Influence of Threshold Values on Edge Detection Performance.

1. Introduction

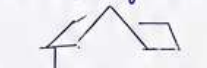
Edge detection identifies points in an image where brightness changes sharply. Most edge detectors (e.g., Canny) use threshold values to decide whether a pixel belongs to an edge or not. The choice of threshold values strongly affects the quality of the result.

2. Thresholding in Edge Detection

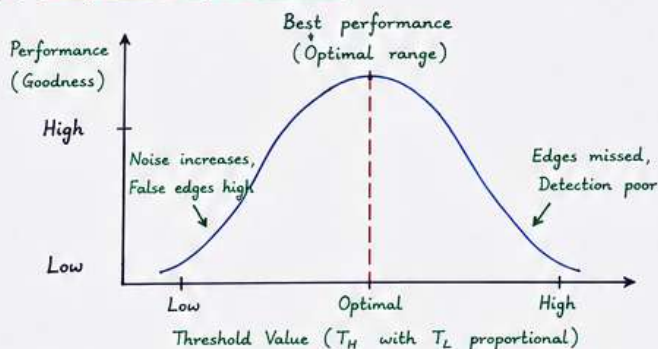
In Canny edge detector, two thresholds are used :

- High threshold (T_H) : Strong edge if gradient $\geq T_H$
- Low threshold (T_L) : Weak edge if $T_L \leq \text{gradient} < T_H$
- $T_L < T_H$. Weak edges connected to strong edges are kept (hysteresis), others are removed.

3. Influence of Threshold Values

Threshold Setting	Effect on Edge Map	Performance Analysis
Very Low T_L and T_H	Many edges detected, including noise 	• High sensitivity • High false positives • Poor localization • Noisy result
Low T_L , Moderate T_H	More edges, some noise removed 	• Good sensitivity • Some false edges remain • Moderate localization
Optimal T_L and T_H	True edges detected clearly 	• Good balance between sensitivity and specificity • Accurate localization • Best performance
High T_L and T_H	Only strong edges detected, many missed 	• Low sensitivity • Missed weak but important edges • Incomplete result
Very High T_L and T_H	Very few or no edges detected 	• Very low sensitivity • Many true edges missed • Poor performance

6. Graphical Representation



8. Conclusion

Threshold values play a crucial role in edge detection performance. Low thresholds detect more edges but include noise, while high thresholds produce cleaner but incomplete results. An optimal threshold range offers the best trade-off between detection accuracy, noise suppression and edge continuity.

Key Point : Proper selection of threshold values is essential for reliable and accurate edge detection.

4. Qualitative Analysis (Example)

Consider a grayscale image (house). Edges detected using Canny with different threshold values.

(Smaller $T_L, T_H \rightarrow$ More edges)

(Larger $T_L, T_H \rightarrow$ Fewer edges)

Thresholds (T_L, T_H)	Edge Map Result	Observation
(10, 20)		• Too many edges • Noise dominant • Poor performance
(20, 40)		• Many edges • Some noise removed • Moderate performance
(50, 100)		• Good edges • Clear object boundaries • Best performance
(80, 150)		• Only strong edges • Weak edges lost • Under-segmentation
(100, 200)		• Very few edges • Important edges missing • Very poor performance

5. Quantitative Influence

As threshold values increase :

- Number of detected edges $\downarrow$ (decreases)
- Noise / False edges $\downarrow$
- Missed edges (False negatives) $\uparrow$
- Edge continuity $\downarrow$
- Localization accuracy $\uparrow$ up to optimal, then $\downarrow$
- Overall detection performance $\uparrow$ up to optimal, then $\downarrow$

7. Guidelines for Choosing Thresholds

- Start with low thresholds and gradually increase.
- Choose thresholds where important edges are detected clearly with minimum noise.
- Use domain knowledge about the image and application.
- Automatic selection methods (e.g., Otsu, entropy) can be used for better results.

Describe Hysteresis Thresholding in the Canny Edge Detector

1. Introduction :

After computing the gradient magnitude and direction and applying non-maximum suppression, the Canny algorithm uses hysteresis thresholding to decide which pixels are true edges. It uses two thresholds: a high threshold (T_H) and a low threshold (T_L) where $T_L < T_H$.

2. Purpose :

- To reduce the effect of noise and weak responses.
- To keep meaningful weak edges that are connected to strong edges.
- To produce thin, continuous and well-localized edges.

3. Steps of Hysteresis Thresholding :

① Classification of pixels

Let G be the gradient magnitude of a pixel.

- Strong edge if $G \geq T_H$
- Weak edge if $T_L \leq G < T_H$
- Non-edge if $G < T_L$

② Edge Tracking by Connectivity

- Start from all strong edge pixels.
- Recursively examine their 8-neighbours.
- If a neighbour is a weak edge, it is included in the edge map and its neighbours are checked.
- Continue until no more weak neighbours are connected to any strong edge.

③ Final Decision

- Pixels marked as strong or connected weak edges $\rightarrow$ Edge (1)
- Remaining pixels $\rightarrow$ Non-edge (0)

4. Algorithm (Pseudo Code) :

Input : Non-maximum suppressed gradient magnitude M , high threshold T_H , low threshold T_L ($T_L < T_H$)

Output : Binary edge map E

1. For each pixel (i,j) in M
2. If $M(i,j) \geq T_H \rightarrow$ mark as STRONG
3. Else if $M(i,j) \geq T_L \rightarrow$ mark as WEAK
4. Else $\rightarrow$ mark as NON-EDGE
5. $E \leftarrow$ zeros (same size as M)
6. For each pixel marked STRONG
7. Push it into stack and set $E(i,j) = 1$
8. While stack is not empty
9. Pop a pixel (p,q)
10. For each 8-neighbour (m,n) of (p,q)
11. If it is WEAK and $E(m,n) = 0$
12. Set $E(m,n) = 1$ and push (m,n) into stack
13. All remaining WEAK pixels not connected to any STRONG pixel are set to 0 in E (suppressed).
14. Return E .

5. Illustrative Example :

(a) Gradient Magnitude (after NMS)

0	0	10	10	0	0
0	50	80	120	60	0
0	60	150	200	90	0
0	40	70	100	50	0
0	10	20	30	10	0
0	0	0	0	0	0

(b) Classification (using $T_H = 100, T_L = 40$)

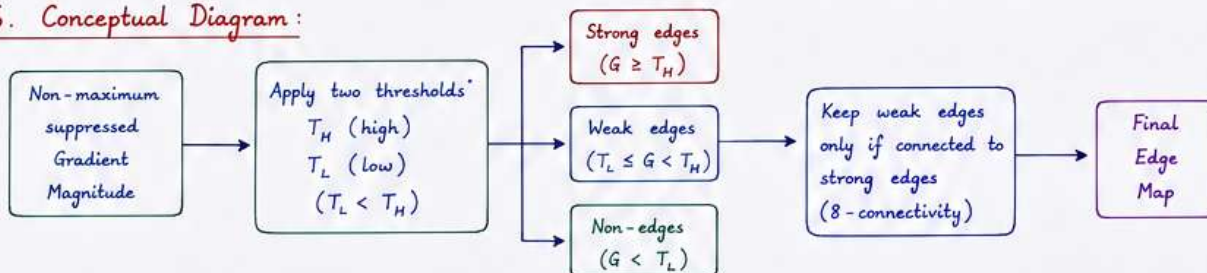
N	N	N	N	N	N
N	W	W	S	W	N
N	W	S	S	W	N
N	W	W	S	W	N
N	N	N	N	N	N
N	N	N	N	N	N

(c) Final Edge Map (after hysteresis)

0	0	0	0	0	0
0	0	1	1	1	0
0	1	1	1	1	0
0	0	1	1	0	0
0	0	0	0	0	0
0	0	0	0	0	0

S - Strong ($\geq T_H$) W - Weak (between T_L and T_H) N - Non-edge ($< T_L$)

6. Conceptual Diagram :



7. Why Hysteresis ?

- A single threshold may break edges or include noise.
- Hysteresis keeps weak responses that are part of true edges and removes isolated noise pixels.
- Results in better edge continuity and fewer false edges.

8. Advantages :

- Improves edge continuity.
- Preserves important weak edges connected to strong edges.
- Reduces noise and false detections.
- Produces thin and well-localized edges.

Key Takeaway : Hysteresis thresholding in the Canny edge detector uses two thresholds (T_H, T_L) and edge tracking by connectivity to keep weak edges only if they are connected to strong edges, resulting in clean, continuous and reliable edge maps.